

20250710 日报

今日工作内容

1. 推进七彩石故障兜底的研究，目前研究进度如下：

1、设计目标：

- 封装成 lib，在使用时代码端只需要导入包即可；
- trpc_go.yaml 中添加七彩石故障时的统一开关；
- 存在多个七彩石时需要按照名称映射（实现思路：备用七彩石的 yaml 文件的文件名与故障前的七彩石名称一致）；
- 实现动态实时监听备用七彩石中配置的更改；

2、设计源码：

i) standbyrainbow 包：

- standbyrainbow.go

```

func init() {
    plugin.Register(pluginName, &Plugin{})
}

// Config-rainbow 配置
type Config struct {
    StandbyRainbow bool          `yaml:"standbyrainbow"`
    Providers       []*rainbow.Config `yaml:"providers"`
}

// Plugin 插件
type Plugin struct {
}

// Type 插件的类型
func (p *Plugin) Type() string {
    return pluginType
}

// Setup 装载插件
func (p *Plugin) Setup(name string, dec plugin.Decoder) error {
    if dec == nil {
        return fmt.Errorf("standbyrainbow config decoder empty")
    }
    cfg := &Config{}
    if err := dec.Decode(cfg); err != nil {
        return err
    }
    // 根据开关配置动态路由
    if !cfg.StandbyRainbow {
        return p.setupPrimaryRainbow(name, dec) // 调用原始七彩石
    }
    // 备用七彩石
    log.Info("Using standby rainbow implementation")
    for i, provider := range cfg.Providers { // 处理所有七彩石配置
        if provider != nil {
            p.setupStandbyRainbow(provider.Name, i)
        }
    }
    return nil
}

```

- standbyKY_config.go

```

// StandbyKV 备用 KV
type StandbyKV struct {
    name      string
    configs   map[string]interface{}
    watcher   *fsnotify.Watcher
    listeners map[string]chan config.Response
    mu        sync.Mutex // watcher 需要上锁
}

// Name 返回七彩石插件名称
func (k *StandbyKV) Name() string {
    return k.name
}

// Get 读取指定七彩石配置
func (k *StandbyKV) Get(ctx context.Context, key string,
    opts ...config.Option) (config.Response, error) {
    val, exists := k.configs[key]
    if !exists {
        return nil, config.ErrConfigNotExist
    }
    yamlData, _ := yaml.Marshal(val)
    return &standbyResponse{
        value: string(yamlData),
        event: config.EventTypePut, // set or update event
    }, nil
}

func (k *StandbyKV) Put(_ context.Context, _ string, _ string,
    _ ...config.Option) error {
    return fmt.Errorf("standby mode does not support put")
}

func (k *StandbyKV) Del(_ context.Context, _ string,
    _ ...config.Option) error {
    return fmt.Errorf("standby mode does not support del")
}

func (k *StandbyKV) initWatcher() error {
    k.mu.Lock()
    defer k.mu.Unlock()
    if k.watcher != nil {
        return nil
    }
    w, err := fsnotify.NewWatcher()
    if err != nil {
        return err
    }
    k.watcher = w
    return nil
}

```

3、使用方法：

- 导入_ "git.woa.com/drewjjli/test/standbyrainbow"包；
- 在 trpc_go.yaml 中的 config-rainbow 下加统一开关 standbyrainbow：

```
config:
  rainbow: # 新增七彩石配置
    standbyrainbow: true # 启动备用七彩石
  providers:
    - name: rainbow
      appid: v1_rb5_dc958896-72e7-4cf3-a097-0b1eb
      group: test
      env_name: test
      address: http://api.rainbow.oa.com:8080
    - name: rainbow_copy
      appid: v1_rb5_dc958896-72e7-4cf3-a097-0b1eb
      group: test2
      env_name: test
      address: http://api.rainbow.oa.com:8080
```

4、功能实现：

a) 基础功能实现：

```
1 config:
2   * personinfo:
3     * name: drewjjli
4     * age: 25
5   * hobbies:
6     * ball: basketball
7     * music: rap
8
```

```
2025-07-10 20:49:20.966 INFO standbyrainbow/standbyrainbow.go:56 Using standby rainbow implementation
Register kv config: rainbow
KVConfigs: map[rainbow:0xc00235fef0]
2025-07-10 20:49:20.967 INFO standbyrainbow/standbyrainbow.go:93 Standby config loaded: standbyrainbow_conf/rainbow.yaml, keys: 1
Register kv config: rainbow_copy
KVConfigs: map[rainbow:0xc00235fef0 rainbow_copy:0xc00289a480]
2025-07-10 20:49:20.967 INFO standbyrainbow/standbyrainbow.go:93 Standby config loaded: standbyrainbow_conf/rainbow_copy.yaml, keys: 1
plugin config-rainbow setup succeed, time elapsed: 495.056µs
2025-07-10 20:49:20.967 INFO trpc-naming-polaris@v0.5.24/naming.go:225 naming-polaris starts to set polaris-go logs config: &{DirPath:./polaris/log Level:default MaxBackups:5 Ma
2025-07-10 20:49:20.967 INFO trpc-naming-polaris@v0.5.24/naming.go:231 naming-polaris starts to set polaris-go logs level: false
plugin selector-polaris setup succeed, time elapsed: 8.745215ms
2025-07-10 20:49:20.976 DEBUG maxprocs/maxprocs.go:48 maxprocs: Updating GOMAXPROCS=16: determined from CPU quota
2025-07-10 20:49:20.977 INFO tnet/server/transport.go:100 --- service: hello.Greeter is using tnet tcp transport, current number of pollers: 1
rainbow/config.go:40 &{drewjjli 25} (basketball rap)
2025-07-10 20:49:20.977 INFO server/service.go:207 process: 3974232, trpc service: hello.Greeter launch success, tcp: 21.6.68.166:8080, serving ...
2025-07-10T20:49:20+08:00 INFO tnet@v0.1.0/tcpserver.go:88 tnet tcp service started, current number of pollers: 1, use tnet.SetNumPollers to change it
```

b) 存在多个七彩石时的名称映射：

- name: rainbow 的七彩石：

```
val, err := config.Get("rainbow").Watch(context.Background(),
"config")
```

```
2025-07-10 20:46:35.214 INFO rainbow/config.go:28 rainbow name: rainbow
2025-07-10 20:46:35.214 INFO tnet/server_transport.go:100 service: hello.Greeter is using tnet tcp transport, current number of pollers: 1
2025-07-10 20:46:35.214 INFO rainbow/config.go:41 &{{drewjlli 25} {basketball rap}}
2025-07-10 20:46:35.214 INFO server/service.go:207 process: 39/1139, trpc service: hello.Greeter launch success, tcp: 21.6.68.166:8000, serving ...
2025-07-10 20:46:35.214 INFO tnet@v0.1.0/tcpserver.go:88 tnet tcp service started, current number of pollers: 1, use tnet.SetNumPollers to change it
```

- name: rainbow_copy 的七彩石：

```
val, err :=
config.Get("rainbow_copy").Watch(context.Background(), "config")
```

```
2025-07-10 20:47:42.179 INFO rainbow/config.go:28 rainbow name: rainbow_copy
2025-07-10 20:47:42.180 INFO tnet/server_transport.go:100 service: hello.Greeter is using tnet tcp transport, current number of pollers: 1
2025-07-10 20:47:42.180 INFO rainbow/config.go:41 &{{drewjlli 27} {basketball rap}}
2025-07-10 20:47:42.180 INFO server/service.go:207 process: 39/2870, trpc service: hello.Greeter launch success, tcp: 21.6.68.166:8000, serving ...
2025-07-10 20:47:42.180 INFO tnet@v0.1.0/tcpserver.go:88 tnet tcp service started, current number of pollers: 1, use tnet.SetNumPollers to change it
```

- c) 动态监听：

```
map[config:map[hobbies:map[ball:basketball music:rap] personinfo:map[age:24 name:drewjlli]]]
Register kv config: rainbow
KVConfigs: map[rainbow:0xc00716480]
2025-07-10 19:59:20.200 INFO standbyrainbow/standbyrainbow.go:91 Standby config loaded: standbyrainbow_conf/rainbow.yaml, keys: 1
plugin config-rainbow setup succeed, time elapsed: 348.856µs
plugin log-default setup succeed, time elapsed: 480.686972ms
2025-07-10 19:59:20.690 INFO trpc-naming-polaris@v0.5.24/naming.go:225 naming-polaris starts to set polaris-go logs config: &{DirPath:./polaris/log Level:default MaxBackups:5 MaxSize:50}
2025-07-10 19:59:20.690 INFO trpc-naming-polaris@v0.5.24/naming.go:231 naming-polaris starts to set polaris-go logs level: false
plugin selector-polaris setup succeed, time elapsed: 8.941953ms
2025-07-10 19:59:20.700 DEBU mapproc/mapproc.go:48 mapproc: Updating GOPAXO@DCS-16: determined from CPU quota
2025-07-10 19:59:20.700 INFO tnet/server_transport.go:100 service: hello.Greeter is using tnet tcp transport, current number of pollers: 1
2025-07-10 19:59:20.700 INFO rainbow/config.go:41 &{{drewjlli 24} {basketball rap}}
2025-07-10 19:59:20.700 INFO server/service.go:207 process: 3918066, trpc service: hello.Greeter launch success, tcp: 21.6.68.166:8000, serving ...
2025-07-10 19:59:20.700 INFO tnet@v0.1.0/tcpserver.go:88 tnet tcp service started, current number of pollers: 1, use tnet.SetNumPollers to change it
2025-07-10 19:59:43.194 INFO rainbow/config.go:41 &{{drewjlli 25} {basketball rap}}
2025-07-10 19:59:47.467 INFO rainbow/config.go:41 &{{drewjlli 26} {basketball rap}}
2025-07-10 19:59:50.280 INFO rainbow/config.go:41 &{{drewjlli 27} {basketball rap}}
```

代码中实现动态监听是基于的 fsnotify 包，它的底层就是利用 epoll 的事件驱动机制，其原理为：

1. fsnotify 包在 Linux 的底层事件驱动原理：
 - a. 初始化阶段：
 - i. 调用 inotify_init1 创建 inotify fd（文件描述符）；
 - ii. 创建 epoll 实例，将 fd 注册到 epoll 树上进行监听；
 - iii. 启动独立协程运行事件循环；
 - b. 事件传递与分发：
 - i. 文件变动（如写入）触发 inotify 事件，内核将事件数据写入 inotify fd 缓冲区；
 - ii. epoll 检测到 fd 可读时，就唤醒阻塞的 epoll_wait；
 - iii. 唤醒阻塞后，通过 read() 读取 inotify fd 的原始事件数据（InotifyEvent 结构体）；
 - iv. 将 InotifyEvent 转换为标准 fsnotify.Event，推送到 Events 供用户程序处理（Channel 类型）；

2. 基于 fsnotify 实现 yaml 文件中配置实时监听的上层步骤：

- a. 调用 `fsnotify.NewWatcher()` 创建监听器；
- b. 用户调用 `watcher.Add(path)` 添加监控路径；
- c. 独立协程 for-select 循环读取 Events 通道；
- d. 从 Events 通道检测到写入操作就处理 yaml 配置文件的更新，即重新读一遍；

今日所遇问题：

1、如何在七彩石故障时程序不更改的前提下不报错且字段切换到备用七彩石中读取配置？

覆盖 config-rainbow 插件的 `Setup()` 函数即可，开关触发就执行备用七彩石逻辑，不触发就执行原先的 `Setup()` 逻辑即可。

2、如何按七彩石 name 名称来实现对备用七彩石的访问和实现多个备用七彩石文件和 `trpc_go.yaml` 中七彩石 name 的映射？

将备用七彩石 yaml 文件名命名与 `trpc_go.yaml` 中七彩石 name 一致。

3、如何实现同一个七彩石中的多个配置项？

将七彩石平台的键放在相应 Value 的最外层，解析的时候直接把最外层解析为配置项的键即可。

```
config:
  personinfo:
    name: drewjjli
    age: 25
  hobbies:
    ball: basketball
    music: rap

config2:
  personinfo:
    name: drewjjli
    age: 28
  hobbies:
    ball: basketball
    music: rap
```

改变 Watch()中的配置项的键名即可：